OncoExprML: Unsupervised and Supervised Machine Learning Analysis of RNA-Seq Tumour Expression Profiles

Ana Silva[1], Andreia Vieira[1], Beatriz Magalhães[1], Mariana Barbosa[1], Salvador Mota[1],

[1] School of Health, Polytechnic Institute of Porto,
Rua Dr. António Bernardino de Almeida, 400, 4200-072 Porto, Portugal
{10170096, 10250928, 10250035, 10210624, 10250931}@ess.ipp.pt

Abstract. RNA-Seq expression profiling enables computational analysis of molecular patterns across tumour types. This work presents OncoExprML, a reproducible machine learning pipeline for analysing a processed RNA-Seq dataset containing 801 samples, 20,531 expression features and five tumour classes. The analysis was first performed using an unsupervised approach based on PCA and K-Means, allowing the evaluation of natural structure in the data without using class labels. Supervised models, including Logistic Regression, Linear SVM, Linear SVM with PCA 50 and k-NN with PCA 50, were then applied for tumour type classification. The best unsupervised configuration used 50 PCA components and achieved ARI = 0.815 and NMI = 0.865. In supervised classification, both Linear SVM and Linear SVM with PCA 50 achieved the best performance, with F1-score macro = 0.995. The final selected supervised model was Linear SVM with PCA 50, since it preserved predictive performance while reducing the dimensionality of the input space. Additional robustness analyses, including train-test comparison, learning curves, feature selection and label-shuffling, suggested no evidence of severe overfitting within this dataset, although external validation would be required to confirm generalizability.

Keywords: RNA-Seq, tumour classification, machine learning, PCA, K-Means, Linear SVM, bioinformatics.

1 Introduction

Cancer is a heterogeneous group of diseases characterized by diverse molecular profiles that vary between tumour types, tissues of origin and biological contexts [1, 2]. This heterogeneity makes cancer classification a complex problem, since tumours that appear similar at a clinical or histological level may show distinct molecular profiles [2, 3]. Molecular data have therefore become increasingly important in oncology, particularly for studying tumour subtypes, molecular signatures and potential

biomarkers, as well as for supporting computational approaches to tumour classification [3, 4].

Gene expression profiling is one of the main strategies used to study molecular activity in biological samples [1, 5]. RNA sequencing, commonly referred to as RNA-Seq, allows large-scale quantification of gene expression levels and produces datasets that can be analysed using computational and statistical methods [1, 6]. In cancer research, RNA-Seq data have been used to compare tumour types, investigate tissue-of-origin patterns and support machine learning approaches for cancer classification [6, 7]. Recent studies and reviews have shown that machine learning can extract discriminative patterns from gene expression profiles and support multi-class tumour classification tasks [1, 6, 8].

A major challenge in gene expression analysis is the high dimensionality of the data [1, 9]. RNA-Seq and microarray datasets commonly include thousands of features, while the number of available samples is often much smaller [1, 10]. This "large p, small n" structure increases the risk of overfitting and makes preprocessing, dimensionality reduction, feature selection and robust evaluation essential in machine learning workflows [9, 11]. Feature selection is particularly relevant in cancer classification because it can reduce computational burden and help identify expression variables with higher discriminative value [10, 12]. However, the biological interpretation of selected features depends on the availability of real gene identifiers, such as gene symbols or Ensembl IDs [10, 13].

Unsupervised learning is useful as an exploratory step because it allows the structure of the dataset to be examined without using class labels [6, 14]. Principal Component Analysis (PCA) is frequently applied to high-dimensional biological data to reduce dimensionality and visualize major sources of variation [14, 15]. K-Means clustering can then be used to group samples according to similarity in a reduced feature space [16]. In cancer expression datasets, this type of analysis can help determine whether tumour samples form natural groups before any supervised model is trained [6, 7].

However, supervised learning has a different objective: it uses labelled samples to learn patterns associated with predefined classes [1, 8]. Logistic Regression is a supervised linear model often used as a simple baseline for classification tasks, including high-dimensional biological datasets [1, 17]. Support Vector Machines (SVM) are also widely used in gene expression classification and are suitable for high-dimensional settings [18, 19]. Meanwhile, k-nearest neighbours (k-NN) is a distance-based classifier whose performance and computational cost can be affected when the number of features is very high [20]. For this reason, dimensionality reduction with PCA can be useful before applying distance-based methods such as k-NN [15, 20].

Reproducibility is another important concern in computational bioinformatics [21]. Machine learning workflows depend on software versions, preprocessing choices, random seeds and execution environments [21, 22]. Containerization tools such as Docker can improve reproducibility by packaging dependencies and allowing analyses to be executed in controlled environments [22]. This is particularly relevant when a project includes both notebooks and executable scripts, since a reproducible pipeline reduces the chance that results depend on a single local setup [22, 23].

Therefore, this work presents OncoExprML, a reproducible pipeline for analysing RNA-Seq tumour expression profiles. The workflow first applies unsupervised learning with PCA and K-Means to investigate whether tumour samples form natural clusters

without using class labels. Then, supervised learning is applied to classify samples into five tumour classes: *BRCA*, *KIRC*, *COAD*, *LUAD* and *PRAD*. Additional analyses include feature selection, model comparison, stratified cross-validation, computational complexity analysis and Docker-based reproducibility.

2 Objectives

The main objective of this work was to develop a reproducible bioinformatics pipeline for the analysis of RNA-Seq tumour expression profiles using both unsupervised and supervised machine learning approaches.

Specifically, this work aimed to explore the structure of the dataset through descriptive analysis and PCA visualization; evaluate whether tumour samples formed natural groups using PCA + K-Means clustering without class labels; and compare the resulting clusters with the real tumour classes using ARI, NMI and contingency tables.

In the supervised stage, the work aimed to train and compare classification models, including Logistic Regression, Linear SVM, Linear SVM with PCA 50 and k-NN with PCA 50. Model performance was assessed using macro-averaged metrics, confusion matrices and stratified cross-validation. Additional analyses were also performed to evaluate the effect of feature selection, the impact of PCA before Linear SVM, and potential overfitting through train-test comparison and learning curves.

Finally, the work aimed to improve reproducibility and portability through Python scripts, exported models, Docker and computational complexity analysis. The complexity analysis combined theoretical Big-O notation with empirical measurements of execution time and approximate RAM usage.

3 Materials and Methods

3.1 Dataset

The dataset used in this work was the "Gene Expression Cancer RNA-Seq dataset" from the UCI Machine Learning Repository [24]. This dataset is described as a random extraction of gene expression profiles from the RNA-Seq HiSeq PANCAN dataset [24]. It contains samples from five tumour classes: *BRCA*, *KIRC*, *COAD*, *LUAD* and *PRAD* [24].

The expression matrix contains 801 samples and 20,531 expression features. Each row corresponds to one tumour sample, while each column corresponds to an expression feature. The labels file contains the tumour class associated with each sample.

The input files used in the pipeline were data.csv and labels.csv. The data.csv file contains the RNA-Seq expression matrix, while labels.csv contains the class label for each sample.

The expression features are identified generically as gene_0, gene_1, ..., gene_20530. Because no gene symbols, Ensembl identifiers or RefSeq identifiers are provided, the selected features cannot be directly interpreted as specific biological genes. Therefore, this work analyses the features as anonymous expression variables rather than biologically mapped genes.

3.2 Data Preprocessing

The expression matrix and labels were loaded using Python and pandas. Before applying machine learning methods, the sample identifiers in data.csv and labels.csv were compared to confirm that the expression profiles and class labels were aligned. This validation step was important because mismatched sample identifiers would associate expression profiles with incorrect tumour classes.

Missing values were checked in both the expression matrix and the labels file. No missing values were detected. All expression features were numeric, allowing their direct use in PCA, clustering and supervised classification models.

Expression features were standardized before PCA, K-Means and supervised model training. Standardization transforms each feature by removing the mean and scaling to unit variance, which is important when algorithms are sensitive to feature scale [9]. This step was performed using StandardScaler from scikit-learn [9].

Preprocessing and modelling steps were organized using scikit-learn pipelines. Pipelines allow transformations and models to be combined sequentially, reducing the risk of applying preprocessing inconsistently across training and evaluation stages [10]. This was particularly important for supervised models, where scaling was fitted only on the training data before being applied to the test set.

3.3 Exploratory Analysis

An initial exploratory analysis was performed to characterize the dataset before model training. The number of samples, number of expression features and distribution of tumour classes were examined. This allowed identification of class imbalance and provided an overview of the structure of the dataset.

The distribution of tumour classes was visualized using a bar plot. Class imbalance can affect the interpretation of accuracy and therefore motivated the use of macro-averaged metrics during supervised evaluation.

PCA was used as an unsupervised dimensionality reduction technique rather than as a classifier [14, 15]. It transforms the original expression features into principal components, which are linear combinations of the original variables and capture the main directions of variance in the dataset. Since PCA is unsupervised, tumour labels were not used to fit the components. Labels were used only to colour the exploratory visualization and to evaluate downstream clustering or supervised classification results.

3.4 Unsupervised Analysis: PCA + K-means

To evaluate whether RNA-Seq expression profiles formed natural groups without using tumour class labels during clustering, an unsupervised analysis was performed.

The unsupervised pipeline combined three steps: standardization, PCA and K-Means clustering. Standardization was applied to ensure that all expression features contributed on a comparable scale. PCA was then used to reduce the dimensionality of the expression matrix before clustering [14, 15]. K-Means was applied in the reduced PCA space to assign each sample to a numerical cluster [16].

Different numbers of PCA components were tested before K-Means: 10, 20, 50 and 100. For each configuration, K-Means was applied with five clusters, corresponding to the number of tumour classes in the dataset. The class labels were not used during the clustering process.

The resulting clusters were evaluated using Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI). ARI measures the similarity between two partitions while correcting for agreement expected by chance [11]. NMI measures the amount of shared information between the predicted cluster assignments and the real tumour labels [12]. These metrics were used only after clustering, for evaluation purposes.

A cluster-class contingency table was also generated. This table compared each K-Means cluster with the real tumour classes and allowed identification of the dominant tumour class within each cluster. This interpretation step was necessary because K-Means produces numerical clusters, not direct tumour class predictions.

3.5 Supervised Analysis

After the unsupervised analysis, supervised models were trained to predict tumour type from RNA-Seq expression profiles. The dataset was divided into training and test sets using an 80/20 stratified split. Stratification was used to preserve the relative proportion of tumour classes in both subsets.

Four supervised approaches were evaluated: Logistic Regression, Linear SVM, Linear SVM with PCA 50 and k-NN with PCA 50. Logistic Regression was included as a simple linear baseline for classification [17]. Linear SVM was evaluated because SVM-based models are commonly used in classification tasks and are suitable for high-dimensional datasets, such as gene expression data [18, 19]. Linear SVM with PCA 50 was added to assess whether dimensionality reduction could preserve classification performance while reducing the number of input variables [14, 15]. k-NN was combined with PCA because nearest-neighbour classification is distance-based and can be affected by high-dimensional feature spaces [15, 20].

For Logistic Regression and Linear SVM, the pipeline included standardization followed by the classifier. For Linear SVM with PCA 50, PCA was inserted between standardization and the classifier. For k-NN, PCA was also applied before classification to reduce dimensionality before distance calculations [14, 15, 20].

Model performance was evaluated using accuracy, precision macro, recall macro and F1-score macro. Macro-averaged metrics were selected because the dataset

contains unequal class sizes and these metrics give equal weight to each tumour class. This prevents performance on larger classes from dominating the evaluation. Confusion matrices were also generated to inspect class-specific errors.

Stratified cross-validation was applied to compare model stability across different data partitions. This step was included to reduce dependence on a single train/test split and to provide a more robust evaluation of model performance.

3.6 Evaluation Metrics

Different evaluation metrics were used according to the type of analysis performed. For unsupervised clustering, Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI) were used because K-Means produces numerical clusters rather than direct tumour class predictions. ARI evaluates the similarity between the clustering solution and the real labels while correcting for agreement expected by chance [11]. NMI measures the amount of shared information between the predicted clusters and the real tumour classes [12]. These metrics were therefore appropriate to assess whether the clusters obtained without using class labels reflected the known tumour types.

For supervised classification, accuracy, precision macro, recall macro and F1-score macro were used. Macro-averaged metrics were selected because the dataset contains unequal class sizes and macro averaging gives equal weight to each tumour class. This prevents performance on larger classes from dominating the evaluation and is particularly relevant in biomedical classification problems with class imbalance [1, 9]. Confusion matrices were also generated to inspect class-specific errors and identify which tumour classes were more frequently confused.

ROC/AUC curves were not used as the main evaluation approach because this work addressed a five-class classification problem rather than a binary classification task. Although multiclass ROC analysis could be performed using a one-vs-rest strategy, this would require separate curves for each class and would be less direct for comparing the models in the context of this study. Therefore, macro-averaged precision, recall and F1-score, together with confusion matrices and stratified cross-validation, were considered more appropriate and interpretable for evaluating supervised model performance.

3.7 Feature Selection

Feature selection and PCA were used as complementary strategies to address the high dimensionality of the RNA-Seq dataset [10, 12-15]. PCA reduces dimensionality by creating new components that are linear combinations of the original features, whereas feature selection keeps a subset of the original expression variables [13-15]. Therefore, PCA was useful for dimensionality reduction and computational efficiency, while feature selection was useful for evaluating whether a smaller number of original anonymous features retained discriminative information [10, 12, 13].

This distinction was important because the dataset contained 20,531 expression features but only 801 samples, a common challenge in gene expression analysis [1, 9].

Reducing the number of variables can help decrease computational burden, reduce noise and limit the risk of overfitting in high-dimensional classification tasks [9–13].

The method used was SelectKBest with ANOVA F-test. SelectKBest selects the features with the highest scores according to a specified scoring function [13]. In this work, f_classif was used as the scoring function because it is suitable for classification problems [13].

Different feature subset sizes were evaluated: 100, 500, 1000 and 5000 features. For each subset size, a Linear SVM pipeline was trained and evaluated. The objective was to determine whether reducing the number of features affected classification performance.

The selected features were reported using their original anonymous identifiers. Since the dataset does not provide a mapping between these identifiers and real gene symbols, the selected features were interpreted as computationally discriminative expression variables rather than biologically validated genes.

3.8 Overfitting and Robustness Assessment

Because the dataset contained many more features than samples, additional analyses were performed to assess potential overfitting and model robustness [1, 9, 11]. First, train and test F1-score macro values were compared for each supervised model. This comparison was used to evaluate whether models performed substantially better on the training set than on unseen test data.

Second, stratified cross-validation was used to evaluate whether model performance remained stable across different data partitions. This reduced the dependence on a single train/test split and provided a more robust estimate of model performance.

Third, a learning curve was generated for the final Linear SVM with PCA 50 model. Training and validation F1-score macro were calculated using increasing training set sizes. This analysis was used to assess whether validation performance improved with more data and whether a large gap existed between training and validation scores.

Finally, a lightweight label-shuffling control was performed. In this control, training labels were randomly shuffled and the model was trained again. The objective was to verify whether model performance decreased when the relationship between expression profiles and tumour classes was destroyed. This analysis was interpreted as a sanity check rather than a formal permutation test.

3.9 Computational Complexity Analysis

An approximate computational complexity analysis was performed to summarize the main computational costs of the pipeline. This analysis was relevant because the dataset contains many more features than samples, with 801 samples and 20,531 expression features, which is a common challenge in gene expression classification and increases

the importance of dimensionality reduction, feature selection and computational efficiency [1, 9-13].

The analysis considered the following notation: n for the number of samples, p for the number of expression features, r for the number of PCA components, q for the number of K-Means clusters, c for the number of tumour classes, i for the number of iterations and F for the number of cross-validation folds.

Standardization and feature selection scale approximately with the size of the expression matrix, $O(n \times p)$, since they operate over samples and features [9, 13]. PCA introduces additional computational cost because it projects the original feature space into a lower-dimensional representation, but it can reduce the dimensionality of downstream analyses [14, 15]. K-Means depends on the number of samples, clusters, PCA components and iterations [16]. k-NN has low training cost but higher prediction cost because distances must be computed between test and training samples, which becomes more relevant in high-dimensional spaces [15, 20]. Cross-validation increases computational cost because model training and evaluation are repeated across multiple folds [1, 9].

A table summarizing the approximate complexity of each main step was generated and exported as complexity_analysis.csv.

In addition to the theoretical Big-O analysis, an empirical computational cost analysis was performed. Following the logic of comparing execution time between algorithms, each supervised pipeline was executed while recording execution time with time.perf_counter(). Approximate RAM usage was also recorded before and after each model execution to estimate spatial cost. Accuracy, precision macro, recall macro and F1-score macro were recalculated in this step only to relate computational cost with predictive performance.

This empirical analysis complemented the theoretical complexity analysis by showing how the models behaved in the project execution environment. The RAM values should be interpreted as approximate estimates rather than exact memory complexity measurements, because Python memory management and the notebook execution environment can influence memory usage between runs.

3.10 Implementation and Reproducibility

The workflow was implemented in Python using pandas, scikit-learn, matplotlib and joblib. The project was organized into separate folders to distinguish input data, generated outputs, notebooks and executable scripts (Table 1). The input/ directory contains the expression matrix and labels, while the output/ directory stores generated tables, figures, trained models and prediction reports. The notebooks/ directory contains the full exploratory and analytical workflow, and the src/ directory contains the executable scripts used to reproduce the main pipeline.

The main script, run_pipeline.py, automates data loading, validation, unsupervised PCA + K-Means analysis, supervised model training, model evaluation and output generation. The predict_sample.py script loads the exported models and analyses an individual sample, reporting both the supervised tumour class prediction and the unsupervised K-Means cluster assignment. The final supervised model, Linear SVM

with PCA 50, and the final PCA + K-Means model were exported using joblib, allowing the models to be reused without retraining.

Docker was used to improve reproducibility and portability of the workflow [21–23]. The project includes a Dockerfile and a requirements.txt file, allowing the computational environment and dependencies to be rebuilt consistently. While local environment managers such as Anaconda or Python virtual environments are useful for managing Python packages, they still depend on the configuration of the host system. Docker provides a more complete containerized environment by packaging the project code, dependencies and execution context together, reducing differences caused by operating systems, package versions or local installations [22, 23].

The Docker image can be built using docker build -t oncoexprml ., and the pipeline can be executed inside the container using docker run --rm -v "$PWD/output:/app/output" oncoexprml. This makes the workflow easier to reproduce on another machine using the same build and run commands, which is particularly relevant in computational bioinformatics workflows [21, 22].

Some exploratory analyses were kept in the notebook rather than added to the automated script. This distinction was maintained because the main script was designed to reproduce the core pipeline, while the notebook contains additional exploratory validation and visualization steps, such as feature selection, label-shuffling, learning curves and empirical computational cost analysis.

Table 1. Project organization.

Directory/File	Purpose
input/	Contains data.csv and labels.csv
output/	Stores generated figures, tables, models and reports
notebooks/	Contains the full exploratory workflow
src/run_pipeline.py	Executes the automated training and evaluation pipeline
src/predict_sample.py	Loads trained models and analyses one sample
Dockerfile	Defines the containerized execution environment
requirements.txt	Lists Python dependencies
README.md	Provides execution instructions

4 Results and Discussion

4.1 Exploratory Analysis

The dataset contained 801 tumour samples and 20,531 RNA-Seq expression features. The class distribution was imbalanced, with *BRCA* being the most represented class and *COAD* the least represented class (Fig. 1). Specifically, the dataset included 300

BRCA samples, 146 *KIRC* samples, 141 *LUAD* samples, 136 *PRAD* samples and 78 *COAD* samples. No missing values were detected in either the expression matrix or the labels, and all expression features were numeric.

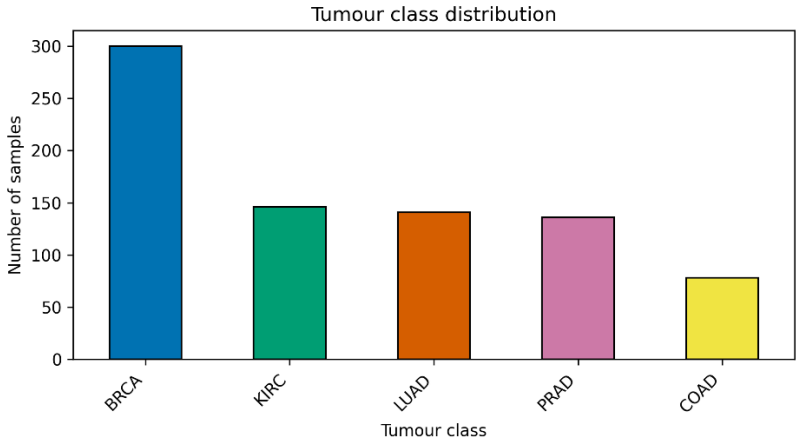


Fig. 1. Distribution of tumour classes in the RNA-Seq dataset. *BRCA* was the most represented class, while *COAD* had the lowest number of samples.

The class imbalance is relevant because accuracy alone may be misleading in datasets where some classes are more represented than others. A model can achieve high accuracy by performing well on majority classes while still performing poorly on smaller classes.

For this reason, macro-averaged precision, recall and F1-score were used in the supervised evaluation. Macro-averaged metrics give equal weight to each class and are therefore more appropriate when class sizes differ, which is common in biomedical datasets and cancer classification tasks [1, 9].

An initial PCA visualization using the first two principal components showed partial separation between tumour classes (Fig 2). PC1 explained approximately 10.5% of the variance, while PC2 explained approximately 8.8%, corresponding to a total explained variance of approximately 19.3%. This relatively low percentage is expected in high-dimensional RNA-Seq data, where relevant variation is distributed across thousands of expression features rather than concentrated in only two dimensions [1, 3].

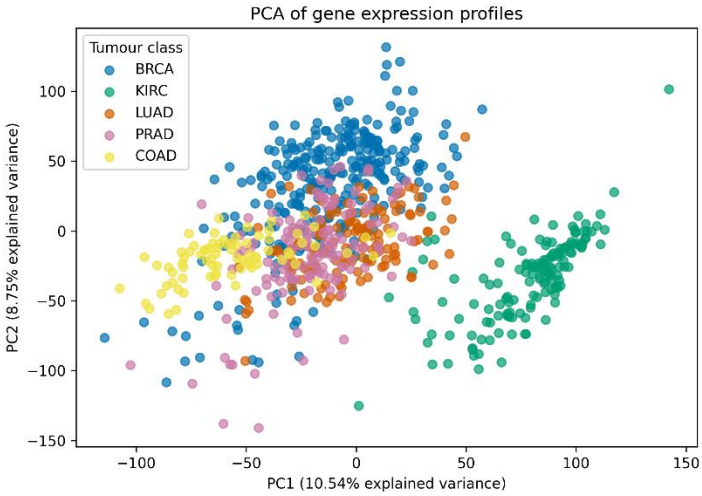


Fig. 2. PCA visualization of RNA-Seq samples using the first two principal components, coloured by tumour class.

The PCA plot should therefore be interpreted as an exploratory visualization rather than a complete separation of tumour classes. The partial separation suggested that the expression profiles contained class-associated structure, but the overlap between some classes also indicated that two-dimensional visualization alone was not sufficient to fully characterize the dataset. This justified the use of both unsupervised clustering and supervised classification. The unsupervised analysis tested whether natural groups could be identified without labels, while the supervised models evaluated whether the full expression profile could be used to accurately predict tumour class.

4.2 Unsupervised Clustering Results

The first modelling stage used PCA followed by K-Means clustering. This analysis was performed before supervised classification because it answers a different question: instead of testing whether labels can be predicted, it evaluates whether the expression profiles form natural groups without giving class information to the algorithm. This is relevant in cancer expression analysis because unsupervised learning can reveal intrinsic structure in molecular data before any classifier is trained [3, 6].

Different numbers of PCA components were tested before K-Means: 10, 20, 50 and 100 components. The objective was to evaluate how the amount of retained dimensional information affected clustering quality (Table 2).

Table 2. Comparison of PCA components for PCA + K-Means clustering.

PCA components	ARI	NMI

10	0.794	0.854
20	0.809	0.860
50	0.815	0.865
100	0.797	0.858

The best clustering configuration used 50 PCA components, achieving ARI = 0.815 and NMI = 0.865. These values indicate substantial agreement between the K-Means clusters and the real tumour classes. Since the labels were not used during clustering, this result suggests that the RNA-Seq profiles contain natural structure associated with tumour type.

The comparison across PCA configurations (Fig. 3) also shows that increasing the number of PCA components does not necessarily improve clustering indefinitely. The configuration with 50 components produced the best ARI and NMI, while 100 components produced slightly lower values. This may occur because retaining more components can also preserve noise or variation that is less relevant for clustering. PCA therefore has two roles in this workflow: it reduces computational cost and can improve clustering by concentrating the dominant sources of variation in a lower-dimensional representation [14, 15].

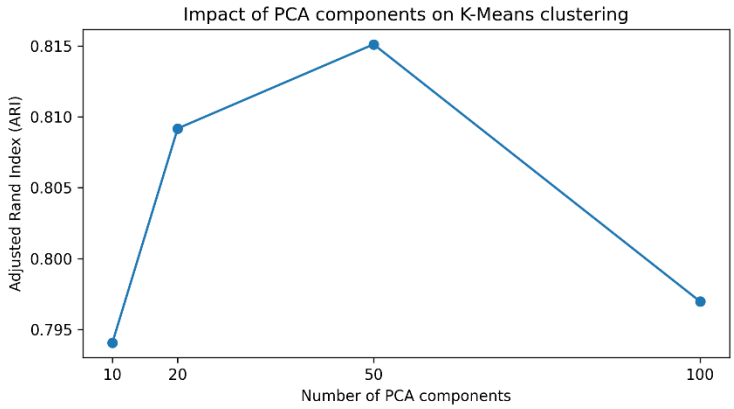


Fig. 3. Impact of the number of PCA components on K-Means clustering performance, measured using ARI.

The cluster-class contingency table (Table 3) provided a clearer interpretation of the K-Means results.

Table 3. Contingency table between K-Means clusters and real tumour classes.

Cluster	*BRCA*	*COAD*	*KIRC*	*LUAD*	*PRAD*
0	0	0	145	0	0
1	253	0	0	2	1
2	0	74	0	0	0
3	0	0	0	0	134
4	47	4	1	139	1

Cluster 0 corresponded almost exclusively to *KIRC*, Cluster 2 to *COAD* and Cluster 3 to *PRAD*. Cluster 1 was dominated by *BRCA,* while Cluster 4 was dominated by *LUAD* but also included several *BRCA* samples. This indicates that *KIRC*, *COAD* and *PRAD* showed clearer separation in the PCA-reduced space (Fig. 4) whereas *BRCA* and *LUAD* showed greater overlap.

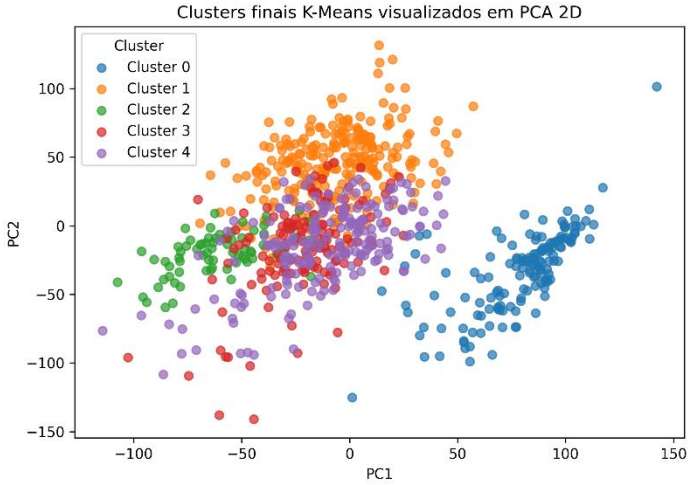


Fig. 4. Final K-Means clusters visualized in two-dimensional PCA space.

The overlap between *BRCA* and *LUAD* is relevant because it appears again in the supervised results, where the only error observed in the final Linear SVM with PCA 50 model involved a LUAD sample classified as *BRCA*. This consistency between independent analyses suggests that the *BRCA*/*LUAD* overlap is not merely an artefact of one method but may reflect similarity between some samples in the expression space.

However, the K-Means results must be interpreted carefully. K-Means does not learn tumour labels and does not directly classify samples into biological classes. It assigns numerical clusters based on similarity, and those clusters only become interpretable after comparison with known labels. Therefore, the unsupervised analysis supports the presence of molecular structure in the dataset, but it should not be presented as a diagnostic classifier [6, 16].

4.3 Supervised Classification Results

After the unsupervised analysis, supervised models were trained to classify tumour samples using RNA-Seq expression profiles. Four approaches were compared: Logistic Regression, Linear SVM, Linear SVM with PCA 50 and k-NN with PCA 50 (Table 4). These models were selected because they represent different classification strategies. Logistic Regression provides a simple linear baseline [17]. Linear SVM is suitable for high-dimensional classification tasks, including gene expression data [18, 19]. Linear SVM with PCA 50 was included to evaluate whether dimensionality reduction could preserve predictive performance while reducing the input space [14, 15, 18, 19]. k-NN is a distance-based classifier, so PCA was applied before k-NN to reduce the effect of high dimensionality on distance calculations [15, 20].

Table 4. Supervised model comparison on the test set.

Model	Accuracy	Precision macro	Recall macro	F1-score macro
Logistic Regression	0.981	0.977	0.973	0.975
Linear SVM	0.994	0.997	0.993	0.995
Linear SVM + PCA 50	0.994	0.997	0.993	0.995
k-NN + PCA 50	0.988	0.994	0.986	0.989

All supervised models achieved high performance, indicating that the expression profiles contain strong discriminative signal for the five tumour classes. Linear SVM and Linear SVM with 50 PCA achieved the best overall test performance, with accuracy = 0.994 and F1-score macro = 0.995 (Fig. 5). This is consistent with previous applications of machine learning to gene expression classification, where linear models and SVM-based approaches often perform well in high-dimensional settings [1, 3, 18, 19].

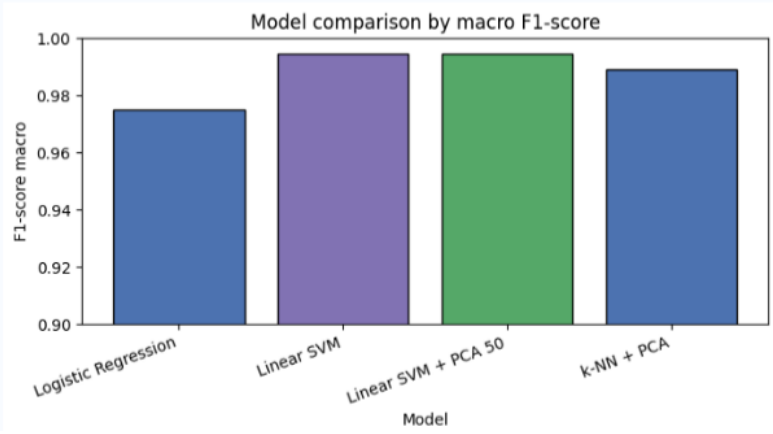


Fig. 5. Comparison of supervised models using F1-score macro.

The inclusion of Linear SVM with PCA 50 was important because it directly addressed the high dimensionality of the dataset. Although the original Linear SVM and the PCA-based Linear SVM achieved the same predictive performance, the PCA-based version used a reduced representation of the data. For this reason, Linear SVM with PCA 50 was selected as the final supervised model.

Logistic Regression also performed strongly, with F1-score macro = 0.975. This suggests that much of the class separation in this dataset can be captured by linear decision boundaries. The k-NN model with PCA 50 achieved F1-score macro = 0.989, confirming that distance-based classification can perform well when dimensionality is reduced before distance calculations [15, 20].

The confusion matrix of the final Linear SVM with PCA 50 model was generated to inspect class-specific errors (Table 5 and Fig. 6). This analysis allowed identification of which tumour classes were more frequently confused by the final supervised model.

Table 5. Confusion matrix of the final Linear SVM with PCA 50 model.

True class	Pred *BRCA*	Pred *COAD*	Pred *KIRC*	Pred *LUAD*	Pred *PRAD*
BRCA	60	0	0	0	0
COAD	0	16	0	0	0
KIRC	0	0	30	0	0
LUAD	1	0	0	27	0
PRAD	0	0	0	0	27

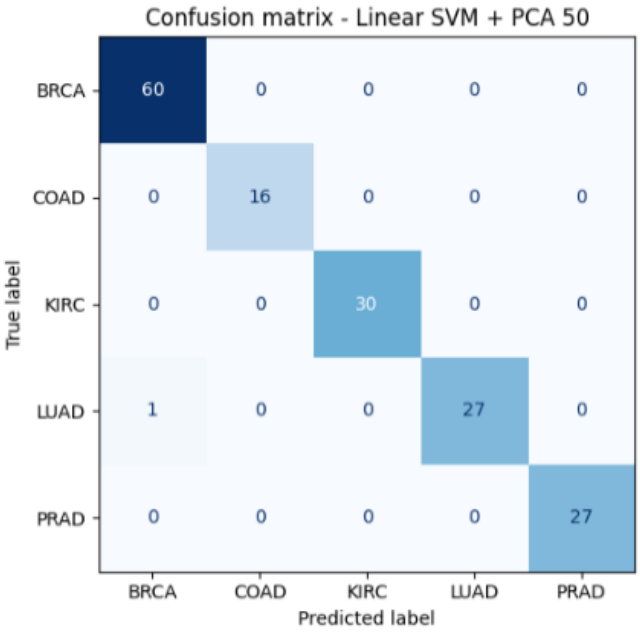


Fig. 6. Confusion matrix of the final Linear SVM model with PCA 50.

The single misclassification between *LUAD* and *BRCA* is coherent with the unsupervised clustering results, where *LUAD* and *BRCA* partially overlapped. This agreement between unsupervised and supervised analyses strengthens the interpretation that some *LUAD* and *BRCA* samples are closer in expression space than other tumour pairs in this dataset.

Stratified cross-validation (Table 6) was used to assess whether model performance was stable across different folds.

Table 6. Stratified cross-validation results.

Model	Accuracy mean	Accuracy SD	Precision macro mean	Recall macro mean	F1-score macro mean	F1-score macro SD
Logistic Regression	0.994	0.006	0.994	0.994	0.994	0.006
Linear SVM	0.999	0.003	0.999	0.999	0.999	0.002
Linear SVM + PCA 50	0.998	0.003	0.998	0.998	0.998	0.003
k-NN + PCA	0.995	0.002	0.997	0.994	0.996	0.002

Stratified cross-validation showed that the supervised models maintained high performance across different folds. Linear SVM achieved the highest mean F1-score macro, while Linear SVM with PCA 50 showed very similar performance with a reduced input representation. This indicates that the strong test performance was not dependent only on a single favourable train/test split.

Nevertheless, these results should be interpreted within the limits of the dataset. The analysis used an internal train/test split and cross-validation, but no independent external validation cohort. Therefore, the results demonstrate strong computational separability within this dataset, but do not establish clinical diagnostic validity.

4.4 k-NN Dimensionality Reduction Analysis

Because k-NN is based on distances between samples, the number of PCA components was evaluated to understand how dimensionality reduction affected both performance and execution time. The tested configurations used 10, 20, 30, 40 and 50 PCA components (Table 7).

Table 7. Effect of PCA components on k-NN performance and execution time.

PCA components	Accuracy	Precision macro	Recall macro	F1-score macro	Time (s)
10	0.981	0.990	0.973	0.981	0.884
20	0.981	0.986	0.973	0.979	0.938
30	0.981	0.986	0.973	0.979	0.903
40	0.988	0.994	0.986	0.989	0.967
50	0.988	0.994	0.986	0.989	1.035

The results show that performance improved when increasing the number of PCA components from 30 to 40, but remained stable between 40 and 50 components (Fig. 7 and Fig. 8). This suggests that very low-dimensional representations may lose part of the discriminative information, while adding more components beyond a certain point provides little additional benefit. The final k-NN model used 50 PCA components, achieving F1-score macro = 0.989.

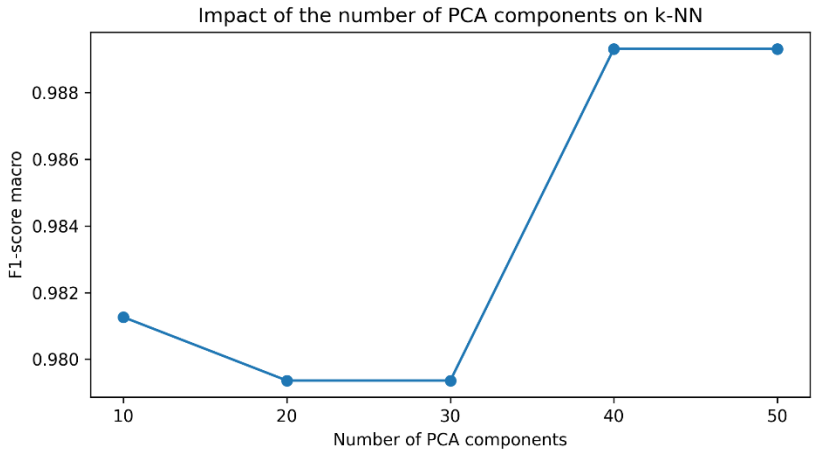


Fig. 7. Impact of PCA components on k-NN F1-score macro.

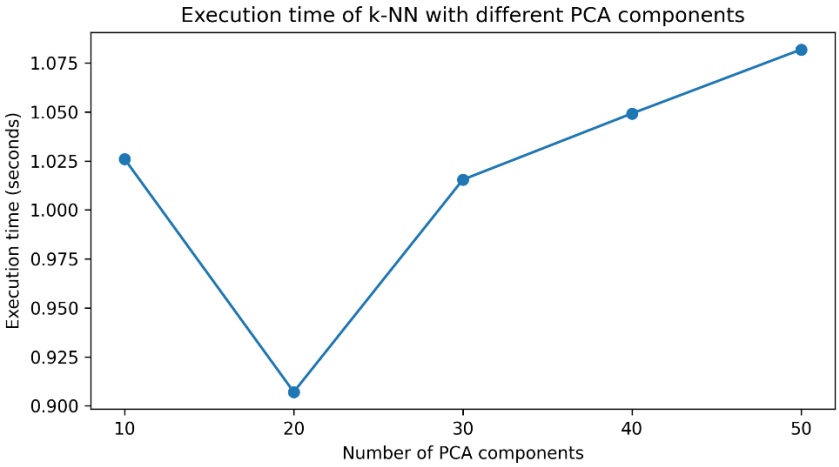


Fig. 8. Impact of PCA components on k-NN execution time.

The execution time increased slightly with more PCA components. This is expected because k-NN has a simple training phase but a more expensive prediction phase, where distances between test and training samples must be calculated. Although the time differences were small after PCA, this analysis supports the decision to reduce dimensionality before applying k-NN. The goal was not only to maintain classification performance, but also to make the distance-based model computationally more manageable [15, 20].

4.5 Feature Selection and Label-Shuffling Control

Feature selection was performed using ANOVA F-test to evaluate whether high classification performance could be maintained using a reduced number of expression features (Table 8 and Fig.9). This analysis is important because high-dimensional expression datasets often contain redundant, noisy or weakly informative variables. Reducing the feature space can lower computational burden and help identify variables with higher discriminative value [10–13].

Feature selection and PCA were used as complementary strategies to address the high dimensionality of the RNA-Seq dataset [10, 12–15]. PCA reduces dimensionality by creating new components that are linear combinations of the original features, whereas feature selection keeps a subset of the original expression variables [13–15]. Therefore, PCA was useful for dimensionality reduction and computational efficiency, while feature selection was useful for evaluating whether a smaller number of original anonymous features retained discriminative information [10, 12, 13].

Table 8. Linear SVM performance after ANOVA feature selection.

Number of features	Accuracy	Precision macro	Recall macro	F1-score macro
100	0.994	0.997	0.993	0.995
500	0.994	0.997	0.993	0.995
1000	0.994	0.997	0.993	0.995
5000	1.000	1.000	1.000	1.000

The results show that strong classification performance was maintained even when using only 100 selected features. This suggests that the discriminative signal is concentrated in a smaller subset of expression variables rather than requiring all 20,531 features. This is consistent with feature selection studies in cancer gene expression classification, where reduced feature subsets can often retain strong predictive performance [10–13].

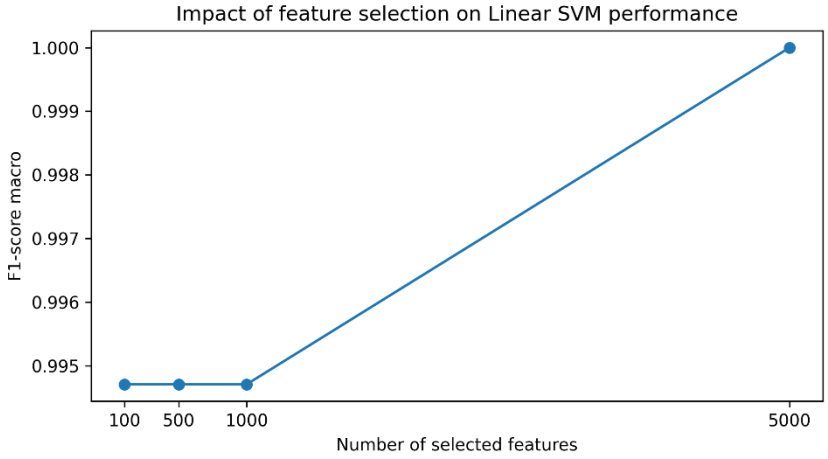


Fig. 9. Effect of ANOVA feature selection on Linear SVM F1-score macro.

The 5000-feature subset reached perfect performance on the selected test split. This should not be overinterpreted. Perfect performance on one internal split does not prove generalizability, especially in high-dimensional datasets. It indicates that the selected features were sufficient to separate the samples in this dataset split, but external validation would be required to determine whether the same performance would hold in independent cohorts.

The top-ranked anonymous features included gene_9175, gene_9176, gene_220, gene_219 and gene_15895, with the highest ANOVA F-scores (Table 9).

Table 9. Top five anonymous features selected by ANOVA F-test.

Feature	ANOVA F-score
gene_9175	3836.253
gene_9176	2978.369
gene_220	2704.445
gene_219	2677.587
gene_15895	2652.678

These features should be interpreted only as computationally discriminative variables. The dataset does not provide real gene symbols, Ensembl IDs or RefSeq identifiers. Therefore, it is not possible to assign biological meaning to these features. This is a major limitation of the dataset. Without a mapping file, claiming that these selected features correspond to specific cancer-related genes or biomarkers would be methodologically incorrect. Future work should use RNA-Seq datasets with real gene identifiers to enable biological annotation, pathway enrichment and comparison with known biomarkers [10, 13].

A lightweight label-shuffling control was also performed in the notebook (Table 10). This control was used to evaluate whether the supervised performance was clearly higher than the performance obtained when the training labels were randomly shuffled (Fig 10). The model trained with real labels achieved F1-score macro = 1.000, while the mean F1-score macro after five label shuffles was 0.197 ± 0.035.

Table 10. Label-shuffling control results.

Real F1-score macro	Mean shuffled F1-score macro	SD shuffled F1-score macro	Number of shuffles
1.000	0.197	0.035	5

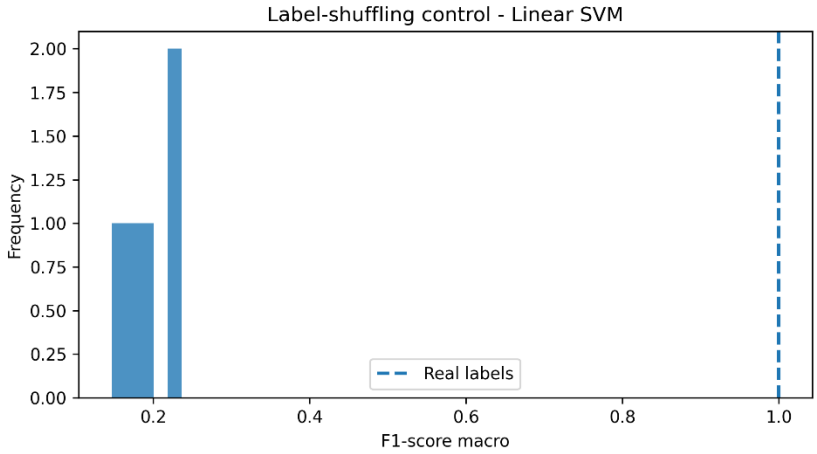


Fig. 10. Label-shuffling control comparing real-label and shuffled-label F1-score macro.

The large decrease in performance after shuffling the training labels supports the interpretation that the model learned class-associated structure rather than arbitrary patterns. This is particularly relevant in high-dimensional gene expression data, where models can appear to perform well if evaluation is not carefully controlled [1, 9, 11].

This analysis should still be described accurately. It was not a formal permutation test with extensive resampling and p-value estimation. It was a lightweight label-shuffling control intended as a sanity check. Therefore, it strengthens the interpretation of the supervised results, but it does not replace independent external validation.

4.6 Overfitting and Robustness Analysis

Because the dataset contains many more features than samples, potential overfitting was specifically considered [1, 9, 11]. Very high classification performance in high-dimensional gene expression datasets should be interpreted carefully, since models may learn patterns that are specific to the dataset rather than generalizable biological signals [9, 11].

To further evaluate this issue, train and test F1-score macro values were compared for the supervised models (Table 11 and Fig. 11). The best models showed very small train-test gaps. Linear SVM and Linear SVM with PCA 50 both had a train-test F1-score gap of 0.005, while k-NN with PCA 50 showed a gap of 0.008. These results do not indicate severe overfitting within this dataset.

Table 11. Train-test comparison

Model	Train F1-score macro	Test F1-score macro	Train-test F1 gap
Logistic Regression	1.000	0.975	0.025
Linear SVM	1.000	0.995	0.005
Linear SVM + PCA 50	1.000	0.995	0.005
k-NN + PCA 50	0.997	0.989	0.008



Fig. 11. Train-test F1-score comparison for supervised models.

A learning curve was also generated for the final Linear SVM with PCA 50 model (Fig. 12 and Table 12). The training score remained high across training sizes, while the validation score increased as more samples were included and approached the training score. This pattern suggests that the model was not only memorizing the training data, since validation performance remained consistently high.

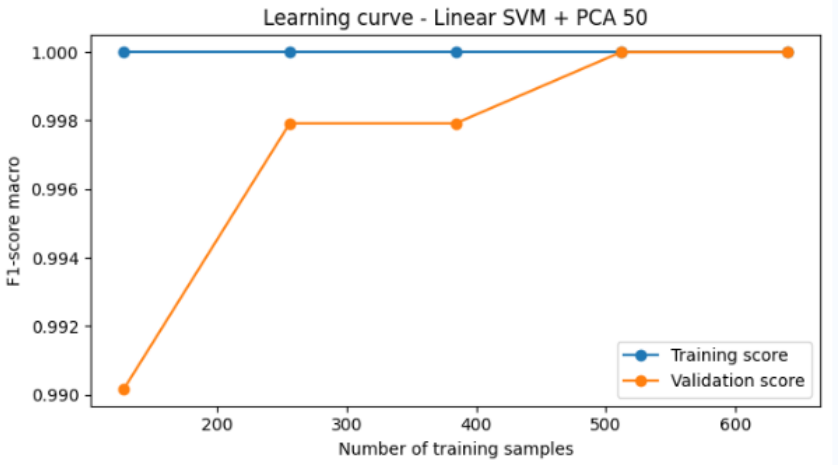


Fig. 12. Learning curve of the final Linear SVM with PCA 50 model. Training and validation F1-score macro were compared across increasing training set sizes.

Table 12. Learning curve values.

Training size	Train F1-score macro mean	Validation F1-score macro mean
128	1.000	0.990
256	1.000	0.998
384	1.000	0.998
512	1.000	1.000
640	1.000	1.000

However, the possibility of overfitting cannot be fully excluded without independent external validation. Therefore, these analyses support the robustness of the model within the current dataset, but external validation would be required to confirm generalizability.

4.7. Complexity and Reproducibility

The computational complexity analysis summarized the approximate cost of the main pipeline steps. This was relevant because the dataset contained 20,531 expression features and only 801 samples, making dimensionality one of the main computational challenges in the workflow.

Table 13 shows that steps operating directly on the full expression matrix, such as data validation, standardization and feature selection, scale with both the number of samples and the number of features. PCA also introduces computational cost, but it reduces the feature space for downstream methods. This trade-off was useful in this

project because K-Means, k-NN and the final Linear SVM with PCA 50 model were applied in a reduced-dimensional representation [14, 15, 20].

The complexity analysis also helps explain the differences between the models. Logistic Regression was used as a simple linear baseline. The original Linear SVM was trained on the full feature space and therefore had a higher computational cost. In contrast, Linear SVM with PCA 50 first reduced the input space from 20,531 features to 50 principal components before classification. k-NN has a relatively simple training phase, but its prediction step requires distance calculations between test and training samples, which is why PCA was applied before k-NN [15, 20].

Table 13. Approximate computational complexity of the main pipeline steps.

Step	Approximate time complexity	Application in this project
Data validation	$O(n \times p)$	Verification of 801 samples and 20,531 features
Standardization	$O(n \times p)$	Mean and standard deviation calculated for each feature
PCA	$O(n \times p \times r)$	Reduction from 20,531 features to PCA components
K-Means	$O(i \times n \times q \times r)$	Clustering in PCA-reduced space
Logistic Regression	$O(i \times n \times p \times c)$	Multiclass linear baseline
Linear SVM	Approximately $O(n \times p)$ to $O(n^2 \times p)$	Supervised classifier using the full feature space
Linear SVM + PCA	$O(n \times p \times r)$ + SVM cost on r components	Final supervised classifier using 50 PCA components
k-NN training	$O(n_train \times r)$	Storage of PCA-transformed training data
k-NN prediction	$O(n_test \times n_train \times r)$	Distance calculations between test and training samples
SelectKBest ANOVA	$O(n \times p)$	Selection of discriminative anonymous features
Cross-validation	$O(F \times$ model cost$)$	Repeated model training and evaluation

Note: In this table, n represents the number of samples, p the number of original expression features, r the number of PCA components, q the number of clusters, c the number of tumour classes, i the number of iterations and F the number of cross-validation folds. The values are approximate and intended to compare the main computational steps of the pipeline.

In addition to the theoretical Big-O analysis, an empirical computational cost analysis was performed (Table 14). This analysis measured execution time, approximate RAM variation and F1-score macro for the supervised models. The objective was to relate predictive performance with computational cost.

Table 14. Empirical computational cost of supervised models.

Model	Execution time (s)	RAM variation (MB)	F1-score macro
Logistic Regression	0.934	20.348	0.975
Linear SVM	18.707	36.809	0.995
Linear SVM + PCA 50	1.190	61.551	0.995
k-NN + PCA 50	1.016	12.398	0.989

The empirical results showed that the original Linear SVM had the highest execution time, requiring 18.707 seconds. In contrast, Linear SVM with PCA 50 achieved the same F1-score macro while reducing execution time to 1.190 seconds (Table 14 and Fig. 13.). This supports the use of PCA before Linear SVM, since dimensionality reduction preserved predictive performance while substantially reducing temporal cost.

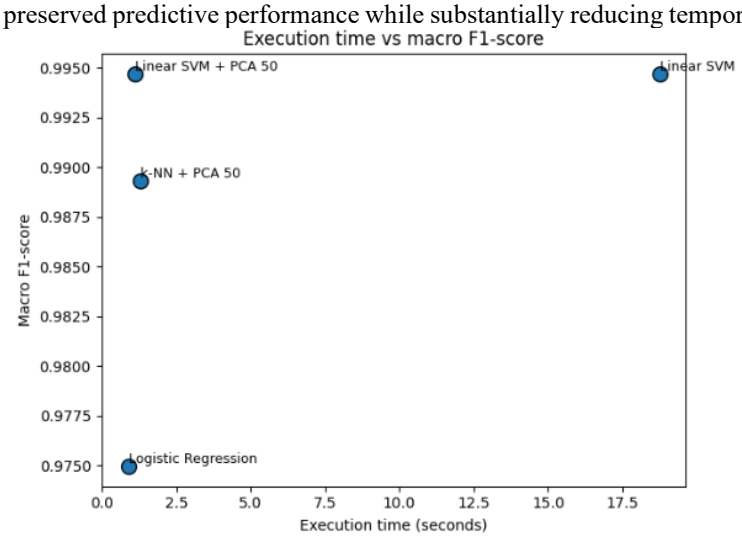


Fig.13. Relationship between execution time and F1-score macro for the supervised models. The plot compares predictive performance with temporal computational cost.

The RAM variation (Table 14 and Fig. 14) values should be interpreted as approximate estimates rather than exact memory complexity measurements. Python memory management and the notebook execution environment can influence memory usage between runs. Therefore, the RAM results were used only as a practical indication of spatial cost, while the theoretical complexity table remained the main formal analysis.

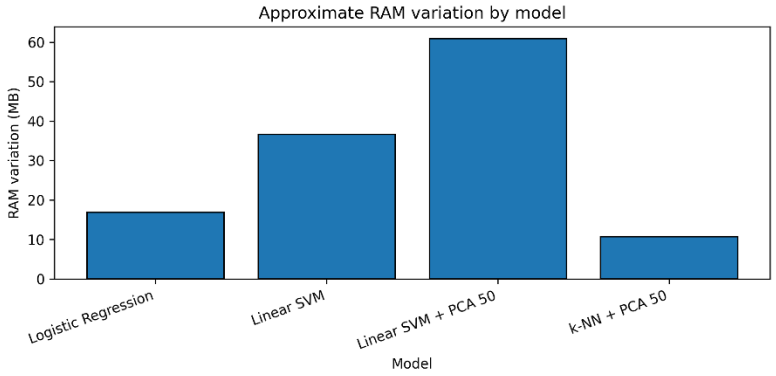


Fig. 14. Approximate RAM variation of the supervised models. RAM values should be interpreted as approximate estimates because memory usage can vary between executions in Python and Jupyter environments.

Reproducibility was addressed by organizing the project into a notebook, executable scripts, exported models and Docker configuration. The notebook contains exploratory analyses and visualizations, while run_pipeline.py reproduces the core pipeline automatically. The predict_sample.py script loads the exported models and analyses an individual sample, reporting both the supervised tumour class prediction and the unsupervised K-Means cluster assignment.

Docker was used to improve reproducibility and portability of the workflow [21–23]. While local environment managers such as Anaconda or Python virtual environments are useful for managing package dependencies, they still depend on the configuration of the host system. Docker packages the project code, dependencies and execution environment into a container, reducing differences caused by operating systems, package versions or local installations [22, 23]. This makes the workflow easier to reproduce on another machine using the same build and run commands, which is particularly relevant in computational bioinformatics workflows [21, 22].

Overall, the complexity and reproducibility analysis showed that the final pipeline combined strong predictive performance with a more efficient reduced-dimensional model and a reproducible execution structure.

5 Future Perspectives

The current version of OncoExprML provides a reproducible computational pipeline for exploring and classifying RNA-Seq tumour expression profiles. Future work could extend this pipeline in several directions, mainly by improving biological interpretability, external validation, robustness and scalability.

A first extension would be to apply the workflow to RNA-Seq datasets containing real gene identifiers, such as gene symbols, Ensembl IDs or RefSeq IDs. In the present dataset, features are represented only as anonymous variables such as gene_0, gene_1 and gene_20530, which prevents direct biological interpretation. Using datasets with

mapped gene identifiers would allow selected features to be linked to real genes, enabling pathway enrichment analysis, biological annotation and comparison with known cancer-related signatures or biomarkers [2, 3, 10, 13].

A second future direction would be to evaluate the pipeline on independent external cohorts. In this work, model performance was assessed using an internal train/test split, stratified cross-validation, train-test comparison, learning curves and a lightweight label-shuffling control. These analyses did not indicate severe overfitting within the current dataset. However, testing the pipeline on independent datasets would provide a stronger assessment of generalizability and would help determine whether the learned expression patterns remain stable across different cohorts, processing pipelines, sequencing platforms or batch effects [1, 2, 9, 11].

Future versions could also expand the biological scope of the analysis. The current workflow focuses on RNA-Seq expression profiles, but tumour biology is influenced by multiple molecular layers. Integrating additional data types, such as genomic variants, DNA methylation, proteomics or clinical metadata, could provide a more complete representation of tumour heterogeneity. Such multi-omics extensions could improve both classification performance and biological interpretation, although they would require careful preprocessing and validation [2, 3].

The machine learning component could also be extended, but this should be done with caution. The current models already achieved high performance, especially the final Linear SVM with PCA 50 model, which suggests that the dataset contains strong discriminative signal even after dimensionality reduction. Therefore, adding more complex models would only be useful if they provided additional interpretability, better external validation performance or improved robustness. Future comparisons could include regularized linear models, tree-based methods or deep learning approaches, but these models should be evaluated carefully to avoid unnecessary complexity and overfitting in high-dimensional datasets [1, 3, 8, 9].

Finally, the reproducibility component could be further improved. The project already includes a notebook, Python scripts, exported models, Docker configuration and a README file. Future versions could add automated tests, command-line arguments, versioned datasets and continuous integration. These additions would make the pipeline easier to maintain, reuse and adapt to other RNA-Seq datasets, supporting reproducible computational bioinformatics practices [21-23].

In general, future work should build on the current pipeline rather than replace it. The most relevant next steps are the use of biologically annotated datasets, validation on independent cohorts and extension towards interpretable multi-omics analysis.

6 Conclusion

This work developed OncoExprML, a reproducible pipeline for the unsupervised and supervised analysis of RNA-Seq tumour expression profiles. The pipeline was applied to a processed dataset containing 801 samples, 20,531 expression features and five tumour classes: *BRCA*, *KIRC*, *COAD*, *LUAD* and PRAD.

The unsupervised PCA + K-Means analysis showed that the expression profiles contain natural structure associated with tumour type. The best clustering configuration

used 50 PCA components and achieved ARI = 0.815 and NMI = 0.865. The cluster-class contingency table showed that *KIRC*, *COAD* and PRAD formed highly specific clusters, while *BRCA* and *LUAD* showed greater overlap. This supports the use of unsupervised learning as an exploratory step before supervised classification in high-dimensional cancer expression datasets [3, 6, 14-16].

The supervised analysis showed strong classification performance across all evaluated models. Linear SVM and Linear SVM with PCA 50 achieved the best performance, both reaching F1-score macro = 0.995. Linear SVM with PCA 50 was selected as the final supervised model because it preserved predictive performance while reducing the dimensionality of the input space. This suggests that the class-discriminative signal was retained in a lower-dimensional representation [14, 15, 18, 19].

Feature selection showed that high performance could be maintained using a reduced subset of expression features. This suggests that the discriminative signal is concentrated in part of the expression profile rather than requiring all 20,531 features. However, because the dataset does not provide real gene identifiers, the selected variables cannot be directly linked to specific genes, pathways or biomarkers. This remains the main limitation of the biological interpretation of the results [10, 13].

The robustness analyses further supported the supervised results. Although the high performance raises a potential concern regarding overfitting, the small train-test F1-score gaps, stable cross-validation results, learning curve behaviour and label-shuffling control did not indicate severe overfitting within this dataset. Nevertheless, independent external validation would be required before making stronger claims about generalizability [1, 9, 11].

The computational complexity analysis showed that PCA was useful not only for dimensionality reduction, but also for improving computational efficiency in downstream models. In particular, Linear SVM with PCA 50 maintained the same F1-score macro as the original Linear SVM while substantially reducing execution time. The empirical RAM values were interpreted only as approximate estimates, since memory usage can vary between executions in Python and Jupyter environments.

The project also demonstrates the value of reproducibility in computational bioinformatics. By combining a notebook, executable scripts, exported models, Docker configuration and documentation, the workflow can be inspected, rerun and extended more easily. Containerized workflows are particularly useful for reducing dependency-related inconsistencies between computational environments [21-23].

Thus, OncoExprML demonstrates that RNA-Seq tumour expression profiles can be analysed through a reproducible machine learning workflow combining unsupervised structure discovery, supervised classification, dimensionality reduction, feature selection, robustness checking and computational complexity analysis. The pipeline should be interpreted as an educational and computational bioinformatics tool, not as a clinical diagnostic system.

References

1. Alharbi, F., Vakanski, A.: Machine Learning Methods for Cancer Classification Using Gene Expression Data: A Review. *Bioengineering* 10(2), 173 (2023). https://doi.org/10.3390/bioengineering10020173
2. Liñares-Blanco, J., Pazos, A., Fernandez-Lozano, C.: Machine Learning Analysis of TCGA Cancer Data. *PeerJ Computer Science* 7, e584 (2021). https://doi.org/10.7717/peerj-cs.584
3. Štancl, P., Karlić, R.: Machine Learning for Pan-Cancer Classification Based on RNA Sequencing Data. *Frontiers in Molecular Biosciences* 10, 1285795 (2023). https://doi.org/10.3389/fmolb.2023.1285795
4. Khalsan, M., Mu, M., Al-Shamery, E.S., Machado, L., Ajit, S., Agyeman, M.O.: A Survey of Machine Learning Approaches Applied to Gene Expression Analysis for Cancer Prediction. *IEEE Access* 10, 27522--27534 (2022). 10.1109/ACCESS.2022.3146312
5. Dalmolin, M., Azevedo, K.S., Souza, L.C., de Farias, C.B., Lichtenfels, M., Fernandes, M.A.C.: Feature Selection in Cancer Classification: Utilizing Explainable Artificial Intelligence to Uncover Influential Genes in Machine Learning Models. *AI* 6(1), 2 (2025). https://doi.org/10.3390/ai6010002
6. Ramroach, S., John, M., Joshi, A.: The Efficacy of Various Machine Learning Models for Multi-class Classification of RNA-seq Expression Data. arXiv preprint arXiv:1908.06817 (2019). https://arxiv.org/abs/1908.06817
7. Younis, H.: Enhancing Cancer Classification from RNA Sequencing Data Using Deep Learning and Explainable AI. *Machine Learning and Knowledge Extraction* 7(4), 114 (2025). https://doi.org/10.3390/make7040114
8. Hashmi, A., Choudhury, A., Alazab, M., et al.: Enhancing Cancerous Gene Selection and Classification for High-Dimensional Gene Expression Data. *Cancers* 16(23), 3913 (2024). https://doi.org/10.3390/cancers16233913
9. scikit-learn developers: StandardScaler. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html
10. scikit-learn developers: Pipeline. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.pipeline.Pipeline.html
11. scikit-learn developers: adjusted_rand_score. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.adjusted_rand_score.html
12. scikit-learn developers: normalized_mutual_info_score. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.metrics.normalized_mutual_info_score.html
13. scikit-learn developers: SelectKBest. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.feature_selection.SelectKBest.html
14. scikit-learn developers: PCA. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.decomposition.PCA.html
15. scikit-learn developers: Decomposition and PCA. *scikit-learn User Guide*. https://scikit-learn.org/stable/modules/decomposition.html
16. scikit-learn developers: KMeans. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.KMeans.html
17. scikit-learn developers: LogisticRegression. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LogisticRegression.html
18. scikit-learn developers: Support Vector Machines. *scikit-learn User Guide*. https://scikit-learn.org/stable/modules/svm.html
19. scikit-learn developers: SVC. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html
20. scikit-learn developers: KNeighborsClassifier. *scikit-learn API Reference*. https://scikit-learn.org/stable/modules/generated/sklearn.neighbors.KNeighborsClassifier.html

21. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., et al.: Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research* 12, 2825--2830 (2011). https://jmlr.org/papers/v12/pedregosa11a.html
22. Alessandri, S., Ratto, M.L., Rabellino, S., Piacenti, G., Contaldo, S.G., Pernice, S., Beccuti, M., Calogero, R.A., Cordero, F., Alessandri, L.: CREDO: A Friendly Customizable, Reproducible Docker File Generator for Bioinformatics Applications. *BMC Bioinformatics* 25, 110 (2024). https://doi.org/10.1186/s12859-024-05695-9
23. Docker Inc.: Docker Documentation. https://docs.docker.com/
24. UC Irvine Machine Learning Repository: Gene Expression Cancer RNA-Seq Data Set. https://archive.ics.uci.edu/dataset/401/gene%2Bexpression%2Bcancer%2Brna%2Bseq